

LAPORAN UJIAN TENGAH SEMESTER ARSITEKTUR DAN PERANCANGAN PERANGKAT LUNAK



DISUSUN OLEH :

**11423048 – JONATHAN MATTHEW NALOM SITORUS
SARJANA TERAPAN TEKNOLOGI REKAYASA PERANGKAT
LUNAK**

**FAKULTAS VOKASI
INSTITUT TEKNOLOGI DEL
SITOLUAMA, 26 MARET 2026**

Case Study 1 : MediTrack - a Digital HealthCare Platform

1. Pemikiran Desain dan Pemilihan Arsitektur

MediTrack dikembangkan dengan arsitektur Hibrid yang menggunakan arsitektur **Monolith** disisi Backend(Laravel). Ditambah dengan pendekatan arsitektur **Modular** dan API-First agar bisa berkomunikasi dengan aplikasi mobile untuk pasien yang dibangun dengan menggunakan Flutter.

1.1 Justifikasi Pemilihan Arsitektur

- Separation of Concerns.
Penerapan arsitektur ini memisahkan lapisan presentasi (Frontend menggunakan Flutter) dengan lapisan logika bisnis dan persistensi data (Backend menggunakan Laravel). Hal ini memastikan bahwa perubahan pada antarmuka pengguna tidak akan mengganggu integritas data medis di sisi server.
- Centralized Data Integrity (Single Source of Truth).
Dalam sistem kesehatan, konsistensi data adalah hal krusial. Dengan struktur Modular Monolith, seluruh catatan medis pasien (EHR) dikelola dalam satu skema basis data terpadu, mencegah terjadinya anomali data (data yang tidak sinkron) saat diakses oleh aktor yang berbeda (Dokter melalui Web dan Pasien melalui Mobile).
- Stateless Communication melalui RESTful API.
Sistem berkomunikasi menggunakan protokol HTTP yang bersifat stateless. Prinsip ini memungkinkan server untuk melayani permintaan secara independen tanpa harus menyimpan status sesi yang membebani memori, sehingga meningkatkan efisiensi penggunaan sumber daya server.

1.2 Identifikasi Trade-Off dan Batasan

- Kecepatan Pengembangan vs Isolasi Layanan: Kami memilih Monolith dibandingkan Microservices untuk memprioritaskan kecepatan pengembangan (Time-to-Market) dan kemudahan koordinasi data selama fase awal. Risikonya adalah ketergantungan antar-modul yang lebih tinggi dibandingkan sistem microservices murni.
- Single Point of Failure: Karena seluruh layanan berada dalam satu unit deployment di sisi backend, kegagalan pada satu modul kritis (seperti modul otentikasi) berpotensi berdampak pada ketersediaan layanan lainnya secara keseluruhan.

1.3 Skalabilitas, Pemeliharaan, dan Kemampuan Perluasan

- **Skalabilitas (Scalability):** Desain API-First memungkinkan skalabilitas horizontal. Jika beban pengguna meningkat, instansi backend dapat direplikasi di belakang Load Balancer tanpa memerlukan modifikasi pada aplikasi mobile, karena klien hanya berinteraksi dengan titik akhir (endpoint) API yang tetap.

- Penggunaan framework Laravel yang berbasis pola Model-View-Controller (MVC) dan Object-Relational Mapping (Eloquent) memudahkan proses debugging dan standarisasi penulisan kode. Struktur folder yang modular memungkinkan tim pengembang melacak alur data medis dengan lebih terstruktur.
- Arsitektur ini mendukung penambahan fitur di masa depan tanpa harus merombak fondasi sistem. Sebagai contoh, integrasi layanan pihak ketiga seperti sistem pembayaran asuransi atau fitur telekonsultasi dapat dilakukan dengan menambahkan modul API baru yang bersifat plug-and-play.

2. Dekomposisi & Pemodelan Sistem MediTrack

Dengan menggunakan Hibrid Modular Monolith, sistem didekomposisi menjadi beberapa modul inti yang saling terintegrasi namun memiliki tanggung jawab yang terpisah secara logika.

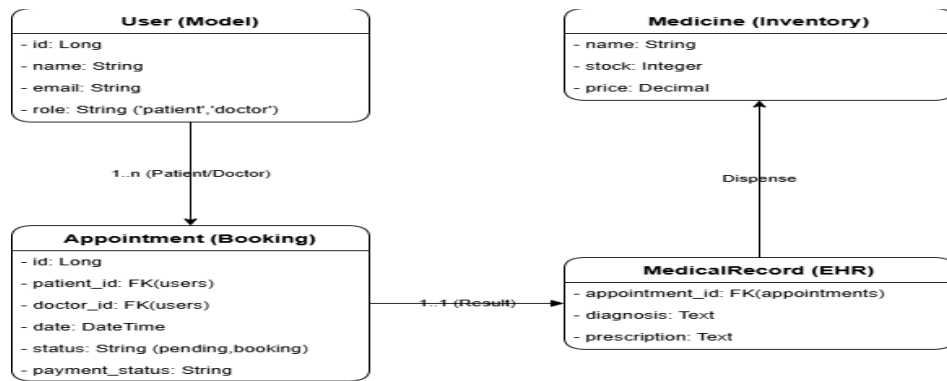
2.1 Daftar Modul dan Tanggung Jawab

Nama Modul	Tanggung Jawab Utama
User & Authentication	Mengelola pendaftaran, autentikasi (Login/Logout), serta manajemen peran (<i>Role</i>) untuk Pasien, Dokter, Apoteker, dan Admin.
Appointment & Scheduling	Mengelola pendaftaran, autentikasi (Login/Logout), serta manajemen peran (<i>Role</i>) untuk Pasien, Dokter, Apoteker, dan Admin.
Electronic Health Record	Mengelola data klinis pasien, termasuk pencatatan diagnosa oleh dokter, riwayat medis, dan hasil laboratorium.
Pharmacy & Inventory	Mengintegrasikan resep dokter dengan layanan apotek, manajemen stok obat, dan status penyerahan obat ke pasien.

2.2 Entitas Utama dan Relasi

User, satu user dapat memiliki banyak Appointment. Appointment, menghubungkan pasien dengan dokter dengan relasi one-to-one. MedicalRecord, bergantung sepenuhnya pada Appointment. Medicine, digunakan dalam modul farmasi untuk menvalidasi ketersediaan resep.

2.3 Pemodelan Sistem



3. Visualisasi High Level Architecture

Sebagai solusi terintegrasi, MediTrack menggunakan pendekatan N-Tier Architecture yang berpusat pada sebuah *Monolith API* yang melayani berbagai jenis klien. Diagram di bawah ini menggambarkan bagaimana komponen-komponen tersebut berinteraksi.

Komponen, Layanan, dan Alur Data.

Sistem terdiri dari tiga lapisan utama yang saling terhubung:

a. Presentation Layer (Client-Side):

- Mobile App (Flutter): Digunakan oleh pasien untuk melakukan *Booking* dan melihat *Riwayat Medis*. Berinteraksi dengan server menggunakan protokol HTTP/REST.
- Web Dashboard (Laravel Blade): Digunakan oleh Dokter dan Admin. Memiliki akses langsung ke logika bisnis di server yang sama untuk performa penginputan data EHR yang cepat.

b. Application Layer (Server-Side - Laravel):

- API Gateway/Interface: Titik masuk tunggal untuk semua permintaan dari aplikasi mobile.
- Modular Logic: Memisahkan fungsi Auth, Penjadwalan, EHR, dan Farmasi dalam struktur folder yang terorganisir.

c. Data Layer:

Database: Menyimpan seluruh status transaksi (Janji Temu, Status Bayar).

b. Batas-Batas Lapisan (Layered Boundaries)

Dalam desain ini, kami menetapkan batas yang jelas untuk menjaga keamanan dan kemudahan pemeliharaan:

- **API Boundary:** Batas antara aplikasi mobile dan server. Semua data yang melewati batas ini harus melalui proses autentikasi (Token-based).
- **System Boundary (Monolith):** Meskipun di dalamnya terdapat berbagai modul (Janji Temu, EHR, Farmasi), mereka berada dalam satu batas layanan Laravel. Hal ini memudahkan koordinasi data antar-modul tanpa perlu jaringan eksternal yang kompleks.

c. Interaksi dengan Sistem Eksternal

Arsitektur MediTrack dirancang agar bersifat terbuka (*Open Integration*) untuk mendukung poin d dan f dalam persyaratan:

1. Payment Gateway (Poin f): Sistem mengirimkan detail tagihan ke *Payment Gateway* (seperti Midtrans/Xendit). Setelah pasien membayar, sistem eksternal mengirimkan *Webhook/Callback* ke server MediTrack untuk secara otomatis mengubah *payment_status* menjadi 'paid'.
2. Pharmacy External API (Poin d): Untuk manajemen stok obat yang lebih luas, modul Farmasi dapat melakukan sinkronisasi dengan API vendor apotek luar guna memastikan ketersediaan obat yang diresepkan oleh dokter dalam EHR selalu akurat secara *real-time*.